NYC Yellow Taxi Data Analysis and Machine Learning Algorithms

Prepared By:
Taha Soylu
Computer Engineering Student

June 8, 2026

1 Understanding the data

If we try to solve a machine learning problem, of course we first need an understandable clean data. To modify the data to give a ML algorithm, we have some methods.However before we diving into these methods, we get rid of NA values and duplicated ones.

1.1 Inter Quantile Range

Many traditional techniques rely on the assumption of "normality" like Z scores or the 3-sigma rule, which become unreliable when dealing with skewed data. The IQR method, however is non-parametric and makes no assumptions about distribution shape, making it inherently more robust. It is ideal for distributions with long tails, skew or multiple peaks.

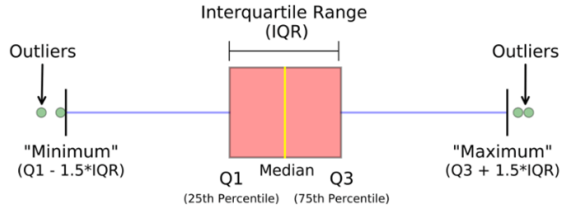

Figure 1: Inter Quantile Range

1.2 Median Absolute Deviation

Normal Z-score uses standard deviation to examine the distribution of the data. MAD is a more robust method. To calculate MAD, you find out how datas deviate from the median and you calculate the meadian od these results.

$$M_i = \frac{0.6745 \cdot (x_i - \tilde{x})}{\text{MAD}}$$

Figure 2: Median Absolute Deviation

1.3 fare amount

"fare amount" is the raw price for the given distance. Here is the process of exploring this column:

The first view of fare amount:

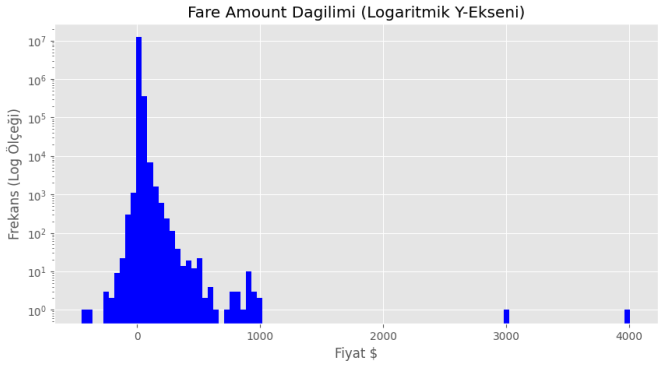

Figure 3: Fare Amount Before

A fare amount cannot be negative:

```
indexes = df[df["fare_amount"] <= 0].index
df.drop(index=indexes, inplace=True)
```

Inter Quantile Range for fare amount: Q1: 6.5, Q3: 13.5, IQR: 7.0 Lower Bound: -4.0, Upper Bound: 24.0

Comment: IQR states that more than 1M rows are outliers. We cannot directly trust the IQR result.

Moreover, I used MAD and MAD stated that 41k rows are outliers. It is more naive compared to IQR.

Both method say a normal fare amount can be outliers. Therefore, I need a quartile result.

```
percentage = 100 * len(df[df["fare_amount"] > 100]) / len(df)
print(f"100 den fazla değer içerenlerin yüzdesi {percentage}")
```

```
100 den fazla değer içerenlerin yüzdesi 0.04360769534076159
```

Figure 4: Fare Amount Percentage

There are few datas more than my quantile, it is a good point to proceed.

An idea to analyze fare amount: Normally we know that fare amount depends on how far we drive the taxi. If I create a new column which shows me fare amount per distance, I can explore where the outlier occurs.

```
df["fare_per_distance"].max()
```

```
np.float64(27500.0)
```

```
df["fare_per_distance"].min()   #!
```

```
np.float64(2.0272790671272644e-07)
```

```
df["fare_per_distance"].quantile(0.999)
```

```
np.float64(62.5)
```

Figure 5: Fare Per Distance

After these results, I cleaned the nonmeaningful data thanks to fare per distance:

```
indexes = df[(df["fare_amount"] > 100) & (df["fare_per_distance"] > 70)].index
df.drop(index=indexes, inplace=True)
```

After this cleaning operations, I deleted the rows which are less than the starting rate:

```
indexes = df[df["fare_amount"] < 1].index
df.drop(index=indexes, inplace=True)
```

After all these data cleaning operations for the fare amout here is the last view:

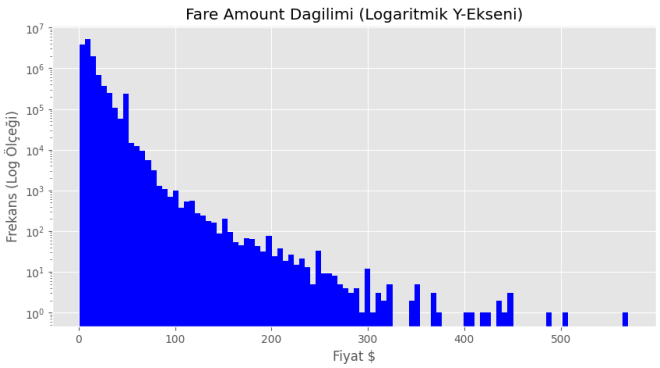

Figure 6: Fare Final View

1.4 extra

Additional fees. These are typically surcharges for rush hour (Rush Hour) or overnight rates (Overnight).

The first view of extra:

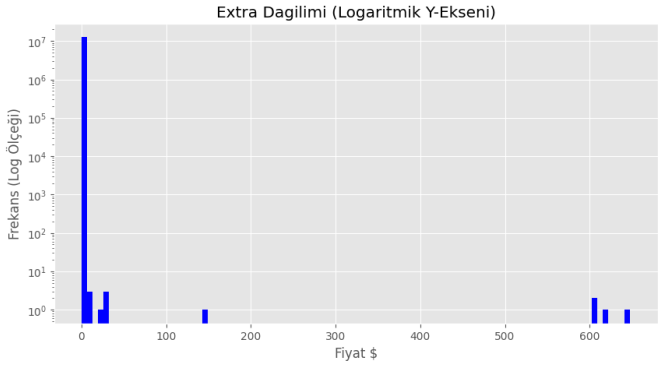

Figure 7: Extra Before

An extra price cannot be negative:

```
indexes = df[df["extra"] < 0].index
df.drop(index=indexes, inplace=True)
```

The most of extra amount distributes on almost zero. If we get the median, we get zero. Again I can use IQR for "extra" amount.

IQR results of "extra": Q1: 0.0, Q3: 0.5, IQR: 0.5 Lower Bound: -0.75, Upper Bound: 1.25

Since I may have normal "extra" amounts after 1.25, I can make my upper bound 2 which results in less data loss. Additionally, I could not use MAD method here, because there are a lot of data equal to zero, my MAD values was 0, which leads to zero division error.

After the data cleaning process:

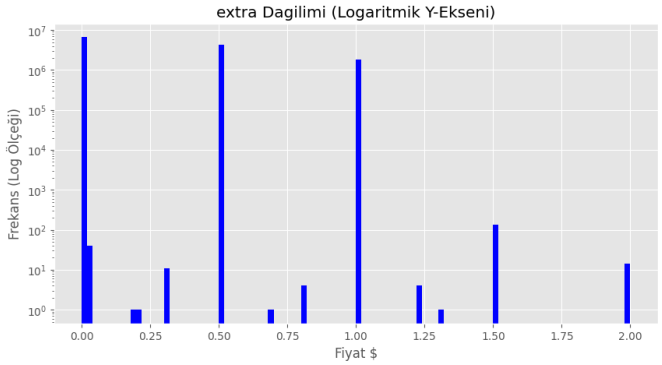

Figure 8: Extra Final

1.5 mta tax

A type of tax for taxi services determined by the government.

The first view of mta tax:

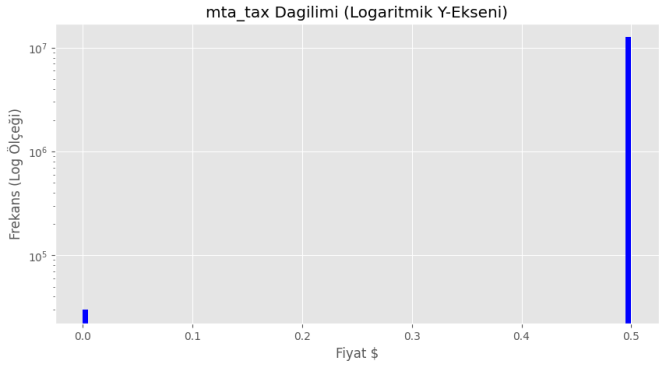

Figure 9: MTA tax

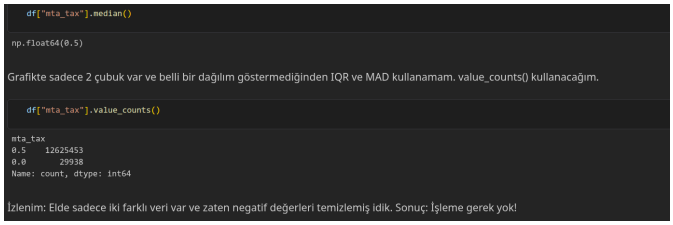

Figure 10: MTA tax notebook

After using "value_counts()" I noticed that there were just two values and there is no need to modify data.

1.6 tip amount

The amount of money the customers tip for the taxi service.

The first view of tip amount:

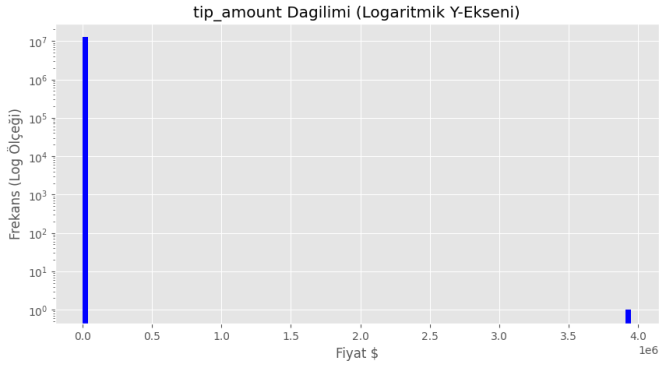

Figure 11: Tip amount first view

A tip amount cannot be negative:

```
indexes = df[df["tip_amount"] < 0].index
df.drop(index=indexes, inplace=True)
```

After exploring the data, I realized that NYC people are not tend to tip. As we see from the graph, the data distributes close to zero point.

IQR Results: Q1: 0.0, Q3: 2.07, IQR: 2.07 Lower Bound: -3.1049999999999995, Upper Bound: 5.174999999999999

Since my plot is right-skewed, the IQR detects some normal tip amounts as outliers like 10 dollars. Need to proceed new techniques.

An idea to analyze tip amount: In real world examples, the customer may tip more if he/she uses the taxi for a long distance. I need to analyze tip amount per fare amount, we do not expect to see a tip amount more than the fare amount.

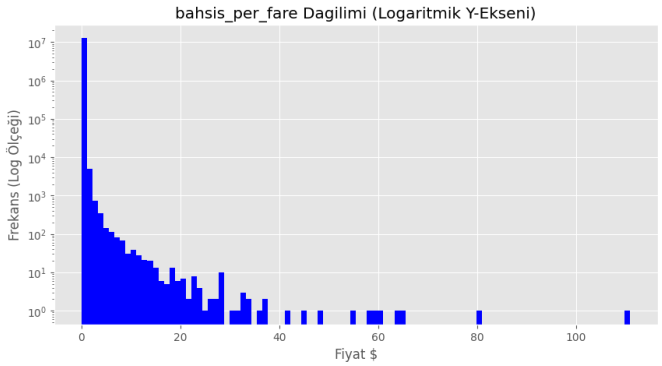

Figure 12: Tip amount per Fare Amount

When we analyze the plot, we can see that there are some absurd values to tip. There are some tip amounts 100 times of its fare amount. I will use MAD to modify my data. The below plot shows the data after cleaning the rows whose MAD values greater than 3.5, which is a rule of thumb for MAD:

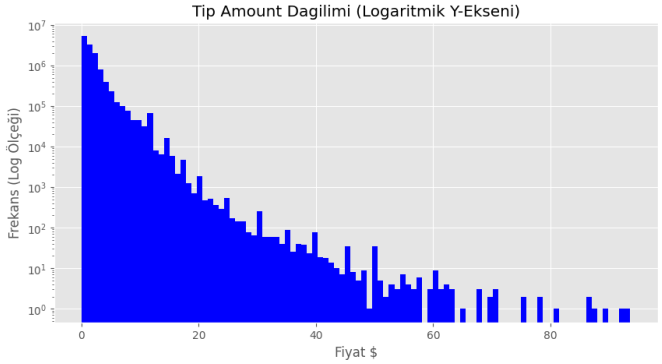

Figure 13: Tip amount final

1.7 tolls amount

A type of price for bridges in NYC.

The first view of tolls amount:

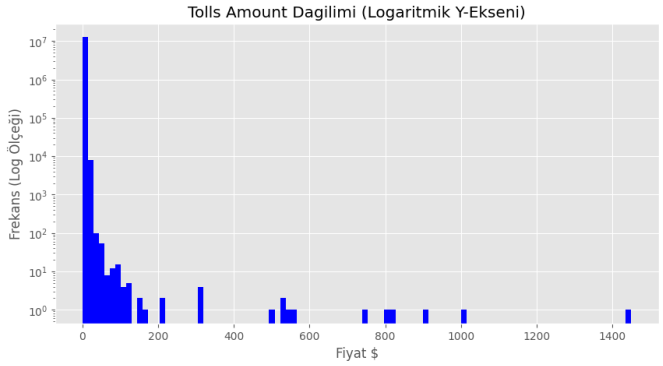

Figure 14: Tolls Amount Before

According to NYC sources a tolls amount can never exceed 200 dollars. I am going to proceed after deleting them:

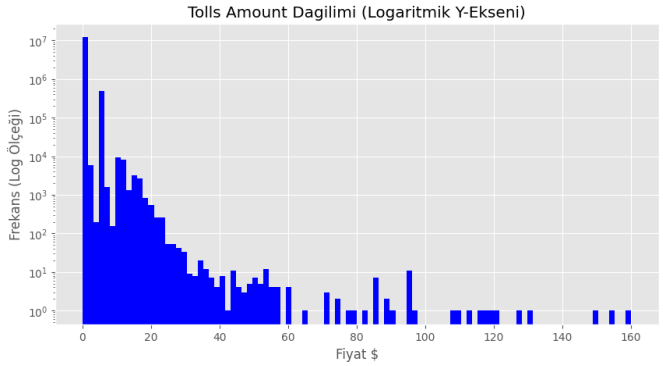

Figure 15: Tolls Amount Middle Phase

IQR Result for tolls amount: Q1: 0.0, Q3: 0.0, IQR: 0.0 Lower Bound: 0.0, Upper Bound: 0.0

Because a lot of data collected on zero point, we cannot get benefit from IQR here. Additionally, the MAD value is zero due to the same problem.

After IQR, MAD and quantile calculations and researches, I decided to make my upper limit 25 dollars.

Figure 16: Tolls Amount Upper Limit

The final view of tolls_amount:

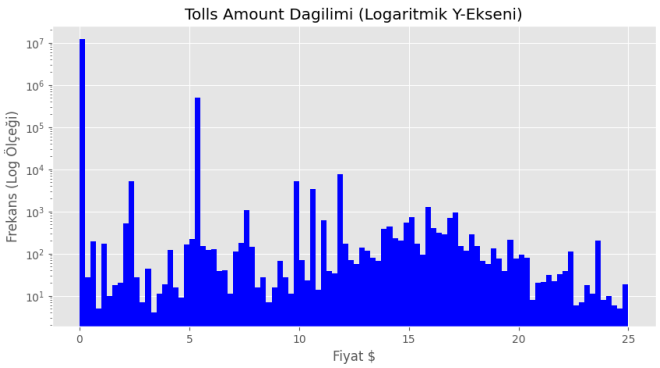

Figure 17: Tolls Amount Final Limit

1.8 duration

In my original data I do not have "duration" column, but I can construct and analyze it. I have both pick up and drop date times. Then:

```
df["duration"] = (df["tpep_dropoff_datetime"] - df["tpep_pickup_datetime"]).dt.
    total_seconds() / 60
```

Duration cannot be negative:

```
indexes = df[df["duration"] <= 0].index
df.drop(index=indexes, inplace=True)
```

The first view of duration:

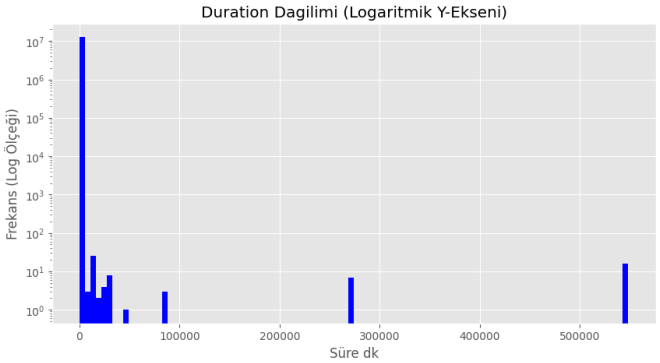

Figure 18: Duration

2500 minutes equal more than 40 hours and it is almost impossible to have a travel like that. After deleting these values I can proceed with IQR analyses.

```
indexes = df[df["duration"] > 2500].index
df.drop(index=indexes, inplace=True)
```

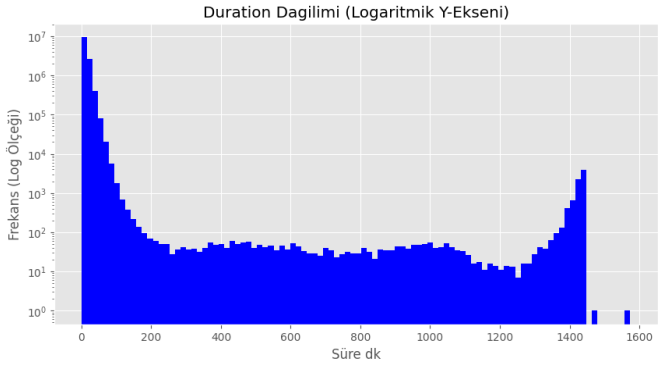

Figure 19: Duration

IQR values for duration: Q1: 6.166666666666667, Q3: 15.833333333333334, IQR: 9.666666666666668 Lower Bound: -8.333333333333336, Upper Bound: 30.333333333333336

IQR states that half million rows of data are outliers.

After MAD calculations and data cleaning, the final view here:

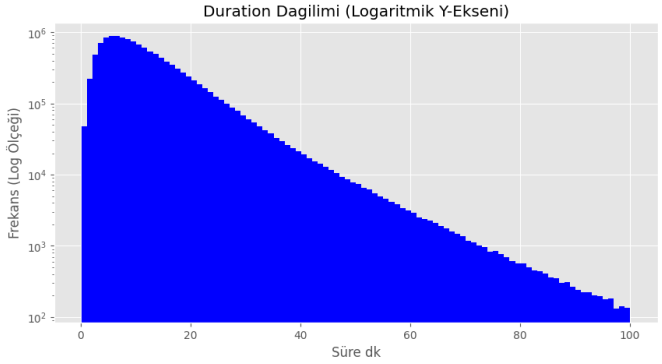

Figure 20: Duration Last View

1.9 Passanger Count

The first view of the passanger count:

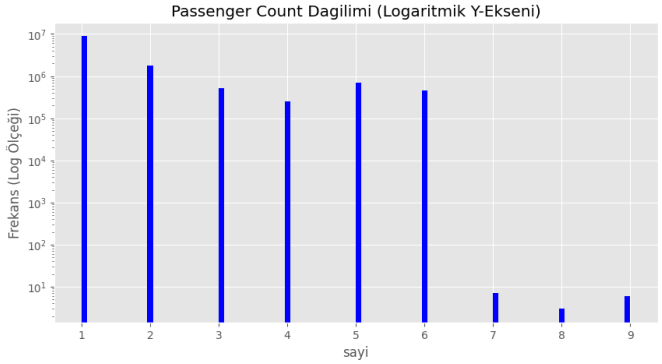

Figure 21: Passanger Count

A vehicle cannot carry more than 6 people and there are few rows (18) which passenger_count value is more than 6.

The final view of passanger_count:

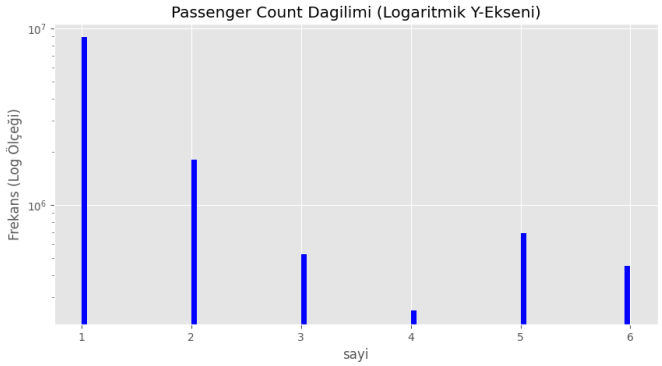

Figure 22: Passanger Count

1.10 Trip Distance

Trip Distance cannot be negative:

```
indexes = df[df["trip_distance"] <= 0].index
df.drop(index=indexes, inplace=True)
```

The first view of trip distance:

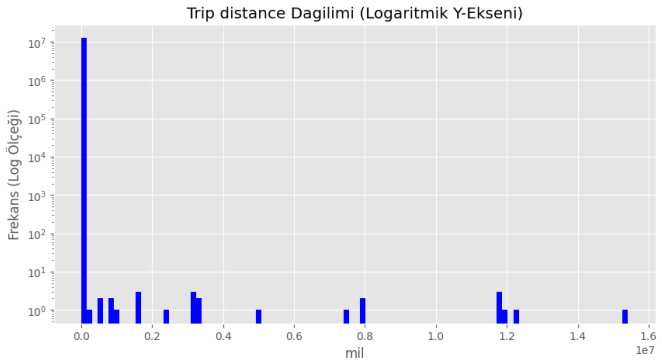

Figure 23: Trip Distance

As we see from the plot, there are some values over the normal range. After cleaning them, I will make IQR calculation:

```
indexes = df[df["trip_distance"] > 20000].index
df.drop(index=indexes, inplace=True)
```

IQR values for trip_distance: Q1: 1.0, Q3: 3.01, IQR: 2.01 Lower Bound: -2.0149999999999997, Upper Bound: 6.0249999999999995

Since NYC people use taxis for short distances, IQR upper limit is too small to crop the data. I could not get an appropriate upper limit using IQR and MAD. Therefore, I use the real world values. Manhattan is 13.5 miles from end to end. If someone were to go back and forth, that would be 27 miles. Let's round it up to 30.

```
indexes = df[df["trip_distance"] > 30].index
df.drop(index=indexes, inplace=True)
```

The final view of trip_distance:

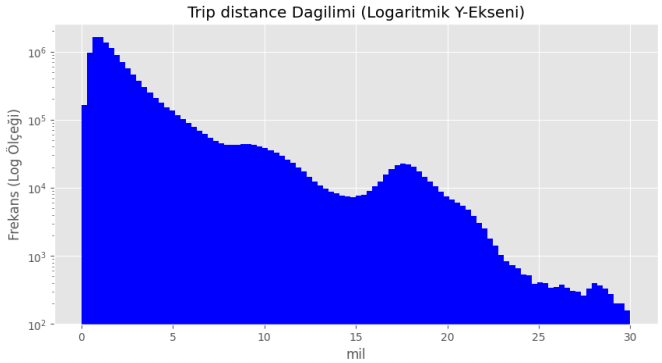

Figure 24: Trip Distance

1.11 speed

Normally, we do not have "speed" column for this dataset. I can construct it using "distance" and "duration" columns. After analyzing this column I can detect the taxis which moves over a normal speed range.

The first view of the speed:

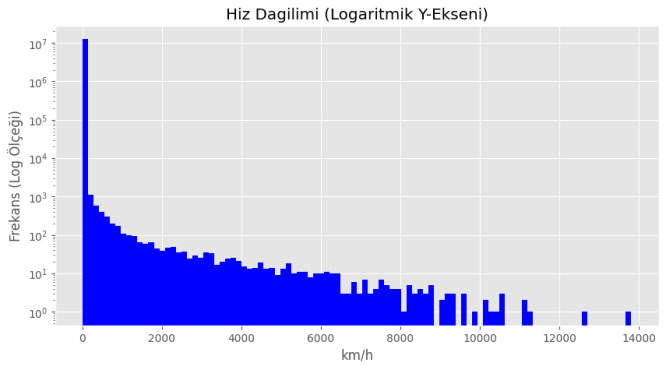

Figure 25: Speed

```
percentage = 100 * (len(df[df["speed"] > 110]) / len(df))
print(f"110 km/h den hizli gidenler", percentage)
```

```
110 km/h den hizli gidenler 0.035199651663676716
```

Figure 26: Speed Limit

A taxi's speed cannot exceed 110 km/h and there is few rows over this speed. I can crop from 110 km/h.

The final view of speed:

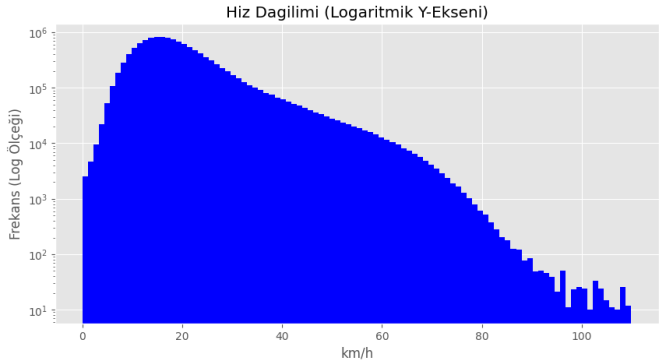

Figure 27: Speed Final

1.12 After data exploring and cleaning

After all these operations, I calculate the trip count per hour and created a new data frame.

1.13 New Data Frame

PULocationID 79 corresponds to the East Village area. This is a neighborhood with a high concentration of bars and young people. It is normal for there to be a large number of taxis in this area at night.

However, on “November 6, 2016,” the clocks were set back in New York. This means that, on that date, midnight actually occurred twice. Therefore, the trip counts in that row should be divided by two.

```
df.loc[df["pickup_hour"] == "2016-11-06 01:00:00", "trip_count"] //= 2
```

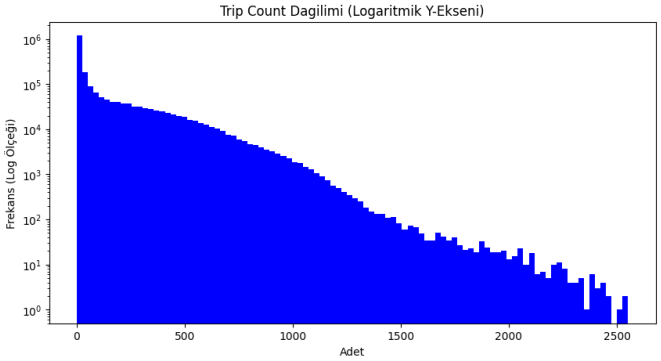

Figure 28: Trip Count

I had previously adjusted the data because the clocks were set back. The same may apply now that the clocks have been set forward, so I need to review two specific dates. ”2015-03-08 02:00:00” and ”2016-03-13 02:00:00”.

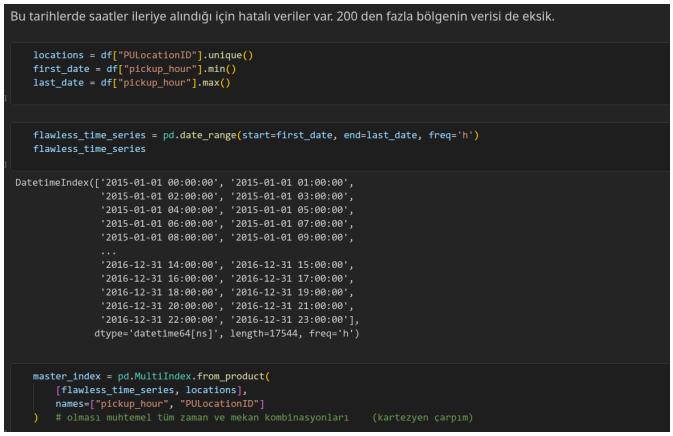

Figure 29: multi_index

To handle absent row problem, I used ”MultiIndex” and ”Reindex”. Pandas automatically created NaN values and I dropped them. Finally, I had a data frame to start Feature Engineering.

2 Feature Engineering

After created the data frame which will be given to the model, we must first design our features. In this section, I constructed new columns. Pandas Datetime facilitiates out job here. I can construct ”hour”, ”day_of_week”, ”month” columns using Pandas Datetime column. Addtionally I added some boolean columns so as to determine whether a specific time is weekend or business hour.

2.1 Cyclical Encoding

In nature, we naturally know that the process occurs cyclical. For example, after 24 the clock ticks 1 and these are close time phases. However, if you think these numbers in a integer line they are far away from each other. We have to make them cyclical.

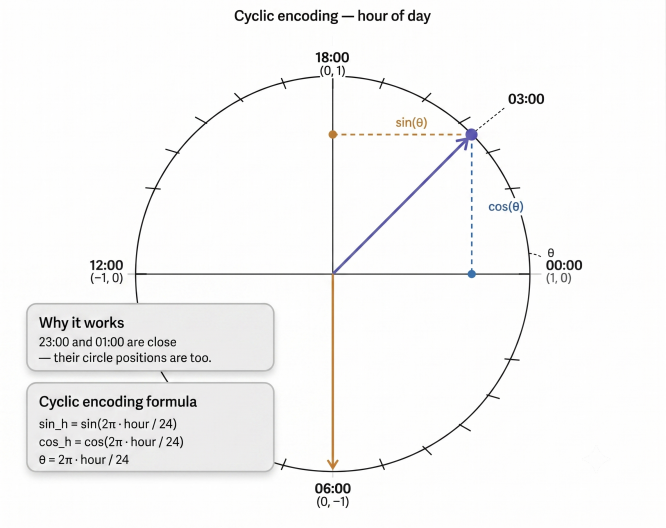

Figure 30: Cycle Encoding

2.2 Lag and Rolling Features

Human behaviour is periodic. If we want our model to predict the future human behaivours we must give it past records. These datas called Lag Features. Additionally, we give the statistical results of past records. A rolling feature calculates a statistical aggregation (mean, sum, standard deviation..) over a moving window of past time steps.

```
#! Short term data analysis
df["rollin_mean_3h"] = past_request.rolling(window=3).mean()    # son 3 saating ortalaması
df["rolling_sum_6h"] = past_request.rolling(window=6).sum() # son 6 saatin toplamı
df["rolling_std_12h"] = past_request.rolling(window=12).std()   # son 12 saatteki standard sapma


#! Long term data analysis
df["rolling_mean_24h"] = past_request.rolling(window=24).mean() # son 24 saating ortalaması
df["rolling_mean_168h"] = past_request.rolling(window=168).mean()   # 1 haftanın ortalaması

#* LAG  # zaten 1 tane shift past_request için yapmıştım
df["lag_24h"] = past_request.shift(23)  # tam 1 gün önceki değer
df["lag_168h"] = past_request.shift(167)    # tam 1 hafta öncesi
```

Figure 31: Rag and Rolling Features

2.3 Correlation Matrix

A correlation matrix is a table showing correlation coefficents between variables.Each cell in the table shows the relationship between two specific variables.

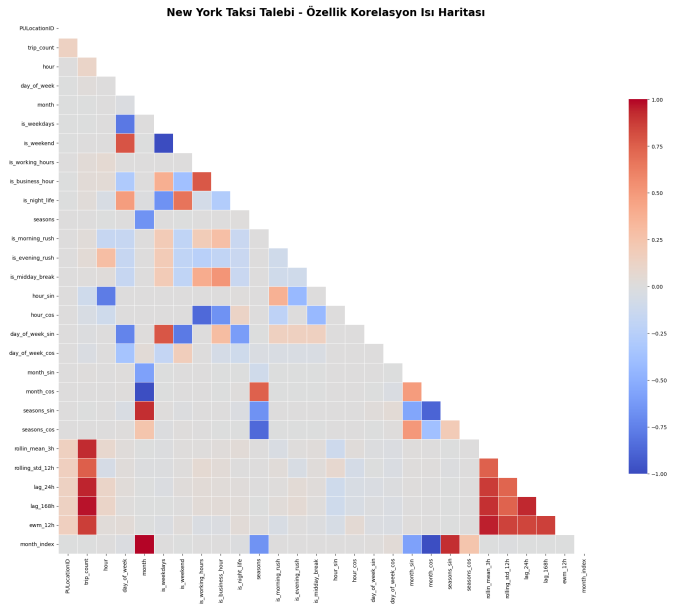

Figure 32: Correlation Matrix

3 Hyperparameter Tuning

Hyperparameter tuning is the process of selecting the optimal values for a machine learning model's hyperparameters. These are typically set before the actual training process begins and control aspects of the learning process itself.

3.1 GridSearchCV

Grid Search is a brute force technique for hyperparameter tuning. It trains the model using all possible combinations of specified hyperparameter values.

```
#! Grid Search için kullanacağım matrix
param_grid = {
    "learning_rate": [0.01, 0.03, 0.05],
    "max_depth": [4, 6, 8],
    "n_estimators": [800, 1200, 1800]
}

grid_search = GridSearchCV(
    estimator=LGBmodel,
    param_grid=param_grid,
    cv=tscv,
    scoring="neg_mean_absolute_error",
    verbose=2,
    n_jobs=1   # modelleri GPU'ya birer birer girmesi için sıraya koydum, yoksa "CUDA Out of Memory" yerim.
)
```

Figure 33: Grid Search

3.2 RandomizedSearchCV

As the name suggests, it picks random combinations of hyperparameters from the given ranges instead of checking every single combination.

```python
param_dist = {
    "learning_rate": uniform(0.01, 0.09),   # learning_rate bu aralıktan seçilecek
    "max_depth": randint(4,10), # max_depth bu aralıktan seçilecek
    "n_estimators": randint(500,2500)   # tree sayısı bu aralıktan seçilecek
}

random_search = RandomizedSearchCV(
    estimator=LGBmodel_random,
    param_distributions=param_dist,
    n_iter=30,  # sadece 30 deneme yap
    cv=tscv,
    scoring="neg_mean_absolute_error",
    verbose=2,
    random_state=42,
    n_jobs=1
)
```

Figure 34: Randomized Search

3.2.1 Bayesian Optimization

Grid Search and Random Search can be inefficient because they blindly try many hyperparameter combinations, even if some are clearly not useful. Bayesian Optimization takes a smarter approach. It treats hyperparameter tuning like a mathematical optimization problem and learns from past results to decide what to try next.

```python
def objective(trial):

    param = {
        "device_type": "gpu",
        "random_state": 42,
        "learning_rate": trial.suggest_float("learning_rate", 0.005, 0.05, log=True),
        "max_depth": trial.suggest_int("max_depth", 5, 8),
        "n_estimators": trial.suggest_int("n_estimators", 2000, 6000),
        "num_leaves": trial.suggest_int("num_leaves", 31, 128)
    }

    tscv = TimeSeriesSplit(n_splits=2)
    optuna_scores = []

    for train_idx, val_idx in tscv.split(x_train):
        x_tr = x_train.iloc[train_idx]
        x_val = x_train.iloc[val_idx]
        y_tr = y_train.iloc[train_idx]
        y_val = y_train.iloc[val_idx]

        LGB_optuna_model = lgb.LGBMRegressor(**param)   # lgb model1
        callbacks = [lgb.early_stopping(stopping_rounds=10, verbose=False)]

        LGB_optuna_model.fit(   # modeli eğittik
            x_tr, y_tr,
            eval_set=[(x_val, y_val)],
            callbacks=callbacks
        )

        # Tahmin yapıp MAE hesabı
        predictions = LGB_optuna_model.predict(x_val)
        mae = mean_absolute_error(y_val, predictions)
        optuna_scores.append(mae)

    return np.mean(optuna_scores)
```

Figure 35: Bayesian Optimization

4 LightGBM

LightGBM addresses the core bottleneck of conventional GBDT — the need to scan all data instances and features to estimate information gain at every split — by introducing two key innovations. First, Gradient-based One-Side Sampling (GOSS) exploits the observation that instances with larger gradients (under-trained instances) contribute disproportionately to information gain; it therefore retains high-gradient instances while randomly discarding only those with small gradients, achieving accurate gain estimation with far less data. Second, rather than sorting raw feature values, LightGBM uses a histogram-based approach that buckets continuous values into discrete bins, drastically reducing memory usage and computation. Together, these techniques allow LightGBM to handle big data — both in the number of instances and features — and speed up training by over 20× compared to standard GBDT, with virtually no loss in accuracy.

LightGBM: A Highly Efficient Gradient Boosting Decision Tree

Guolin Ke[1], **Qi Meng**[2], **Thomas Finley**[3], **Taifeng Wang**[1],
Wei Chen[1], **Weidong Ma**[1], **Qiwei Ye**[1], **Tie-Yan Liu**[1]
[1]Microsoft Research [2]Peking University [3] Microsoft Redmond
[1]{guolin.ke, taifengw, wche, weima, qiwye, tie-yan.liu}@microsoft.com;
[2]qimeng13@pku.edu.cn; [3]tfinely@microsoft.com;

Abstract

Gradient Boosting Decision Tree (GBDT) is a popular machine learning algorithm, and has quite a few effective implementations such as XGBoost and pGBRT. Although many engineering optimizations have been adopted in these implementations, the efficiency and scalability are still unsatisfactory when the feature dimension is high and data size is large. A major reason is that for each feature, they need to scan all the data instances to estimate the information gain of all possible split points, which is very time consuming. To tackle this problem, we propose two novel techniques: *Gradient-based One-Side Sampling* (GOSS) and *Exclusive Feature Bundling* (EFB). With GOSS, we exclude a significant proportion of data instances with small gradients, and only use the rest to estimate the information gain. We prove that, since the data instances with larger gradients play a more important role in the computation of information gain, GOSS can obtain quite accurate estimation of the information gain with a much smaller data size. With EFB, we bundle mutually exclusive features (i.e., they rarely take nonzero values simultaneously), to reduce the number of features. We prove that finding the optimal bundling of exclusive features is NP-hard, but a greedy algorithm can achieve quite good approximation ratio (and thus can effectively reduce the number of features without hurting the accuracy of split point determination by much). We call our new GBDT implementation with GOSS and EFB *LightGBM*. Our experiments on multiple public datasets show that, LightGBM speeds up the training process of conventional GBDT by up to over 20 times while achieving almost the same accuracy.

Figure 36: LightGBM

5 Support Vector Machine

A Support Vector Machine (SVM) is a supervised machine learning algorithm used for both classification and regression, which works by finding the optimal hyperplane — the decision boundary that maximizes the margin between the closest data points (support vectors) of each class. Classification is performed by computing the dot product of an input vector x with the weight vector w, where a point is labeled positive if w·x + b 1 and negative if w·x + b 1. When data is not linearly separable, SVM applies the kernel trick, which maps data into a higher-dimensional space where linear separation becomes possible, with different kernels capturing different relationships. The key advantages of SVM include its ability to maximize the margin (making it robust to new data), handle high-dimensional datasets, and resist overfitting by focusing only on support vectors — however, it comes with notable disadvantages: it is computationally slow on large datasets, memory-intensive due to storing support vectors, difficult to tune (requiring careful selection of kernel, C, and gamma), sensitive to noisy and overlapping data, and generally hard to interpret especially with non-linear kernels.

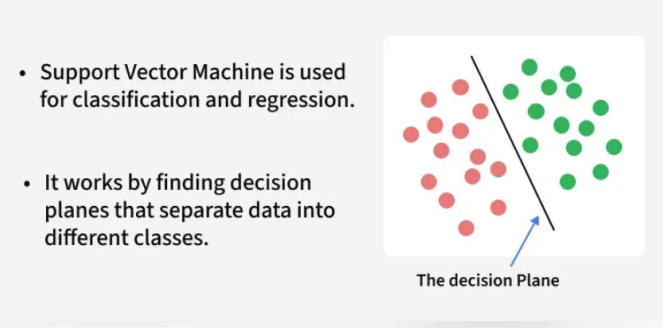

Figure 37: Support Vector Machine

6 Explainable AI

6.1 LIME

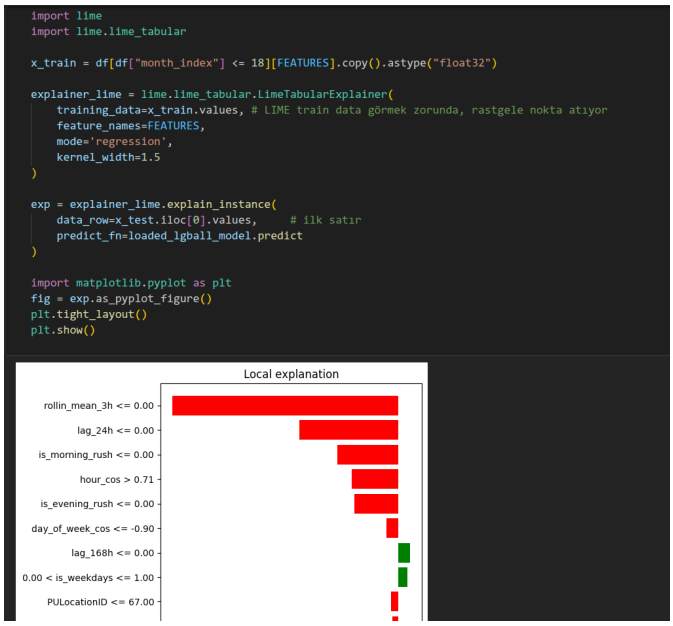

Figure 38: LIME

LIME (Local Interpretable Model-agnostic Explanations) is a model-agnostic interpretability technique that explains the predictions of any black-box ML model locally by generating points sampled from a normal distribution around a reference point, assigning weights to those samples based on their closeness to the chosen point, and training a simple weighted Ridge Regression (f(y) = Kx) on top — making the local behavior of the complex model interpretable through a linear approximation. The optimization goal is to remove the "Dense" constraint while keeping the model locally faithful. Related local methods include: Kernel LIME, which uses a kernel width to provide the neighborhood; Lasso (L1 regularization), which enforces sparsity by setting some coefficients to zero; and ShaLit (Stability), which combines generalization and stability properties. The core problem with LIME is that it relies on a kernel/neighborhood to define the local region, and therefore the MLE (Maximum Likelihood Estimation) relies on the local neighborhood as well — making the choice of kernel and its width critical to the quality of the explanation.

6.2 SHAP

Figure 39: SHAP

SHAP (SHapley Additive exPlanations) is a model-agnostic interpretability method rooted in cooperative game theory that, unlike LIME, calculates the exact mathematical fair contribution of every single feature to a specific prediction. The analogy maps directly: the game is the ML model making a prediction, the players are the features (e.g. Age, Location, Time, Day), and the payout is the difference between the base value (the average prediction over the whole dataset) and the final prediction for that specific instance. To ensure fairness, SHAP computes each feature's marginal contribution by simulating every possible order in which features can "enter the model" — for 3 features (A, B, C) this yields $3! = 6$ permutations — and averages the contribution across all of them. This also accounts for synergy: feature A might contribute nothing alone, but combined with B it may significantly improve the prediction, and SHAP correctly captures this interaction by evaluating features across all possible coalitions rather than in isolation.